

# Selenium Automation Framework — Socratic Requirements Questionnaire

Purpose: Gather requirements before implementation of a scalable, data-driven Selenium Java automation framework. Answer one question at a time in the actual architecture discussion. This document contains the complete question bank and multiple-choice options so requirements can be reviewed upfront.

**Target technology baseline:** Selenium WebDriver + Java + TestNG + Maven + Page Object Model + data-driven testing. The final architecture should be based on the user's selected answers rather than assumptions.

## 1. Application scope

- A. Single web application — One application/project; simplest architecture.
- B. Multiple modules in one web application — One application with several business modules.
- C. Multiple web applications — Framework supports different applications/projects.
- D. Enterprise reusable framework — Designed as a common automation platform across projects.
- E. Other — Describe your scope.

## 2. Application architecture

- A. Traditional server-rendered web application.
- B. SPA — React/Angular/Vue or similar single-page application.
- C. Hybrid — Combination of server-rendered and SPA pages.
- D. Unknown — Application technology will be confirmed later.

## 3. Browser coverage

- A. Chrome only — Optimize primarily for Chrome.
- B. Chrome + Edge — Common enterprise combination.
- C. Chrome + Edge + Firefox — Broader desktop coverage.
- D. Chrome + Edge + Firefox + Safari — Maximum desktop browser coverage.
- E. Other — Specify required browsers.

## 4. Browser versions

- A. Latest stable only — Always execute against the latest browser.
- B. Latest + previous major version — Basic backward compatibility.
- C. Specific organization-supported versions — Versions are controlled by the project.
- D. Dynamic/browser-matrix based — Versions are supplied through configuration/CI.

## 5. Operating systems

- A. Windows only — Local/CI execution on Windows.
- B. Linux only — Typical CI/server environment.
- C. Windows + Linux — Cross-platform execution.
- D. Windows + Linux + macOS — Broad cross-platform support.

## 6. Test types

- A. Functional UI tests only.
- B. Functional + regression testing.
- C. Functional + regression + smoke/sanity.
- D. Functional + regression + smoke/sanity + integration/end-to-end.
- E. Other — Specify test types.

## **7. Test organization**

- A. TestNG classes grouped by feature/module.
- B. TestNG groups/tags such as smoke, regression, sanity.
- C. Both feature organization and TestNG groups.
- D. BDD-style organization with Cucumber scenarios.
- E. Other.

## **8. Test framework**

- A. TestNG — Primary test runner.
- B. JUnit — Use JUnit instead of TestNG.
- C. TestNG + Cucumber — TestNG controls execution and Cucumber provides BDD.
- D. Other — Specify.

## **9. Java version**

- A. Java 8 — Legacy compatibility.
- B. Java 11 — Long-term enterprise baseline.
- C. Java 17 — Modern LTS baseline.
- D. Java 21 — Newer LTS baseline.
- E. Other — Specify version.

## **10. Build tool**

- A. Maven — Standard Maven lifecycle and dependency management.
- B. Gradle — Alternative build/dependency management.
- C. Other — Specify.

## **11. Selenium version strategy**

- A. Latest stable Selenium.
- B. Pinned Selenium version controlled by the project.
- C. Automatically managed/upgraded Selenium dependencies.
- D. Other.

## **12. Page Object Model approach**

- A. Classic Page Object Model — One class per page.
- B. Page Objects + reusable components — Pages plus common widgets/components.
- C. Page Objects + component objects + business flows — Strong abstraction for scalability.
- D. Other architecture — Specify.

## **13. Page object design**

- A. Page classes contain locators and UI actions only.
- B. Page classes contain locators, UI actions, and small validations.
- C. Separate page/component layer and business-flow/service layer.
- D. Other.

#### **14. Locator strategy**

- A. Stable IDs/data-testid preferred.
- B. CSS selectors preferred.
- C. XPath allowed when necessary.
- D. Combination with a locator priority rule.
- E. Other organization-specific strategy.

#### **15. Synchronization strategy**

- A. Explicit waits only — WebDriverWait/ExpectedConditions.
- B. Explicit waits + custom reusable wait utilities.
- C. Fluent waits + explicit waits based on component behavior.
- D. Other.

#### **16. Dynamic element handling**

- A. Reusable wait utilities.
- B. Custom conditions for application-specific states.
- C. Component-level synchronization.
- D. Combination of A+B+C.

#### **17. Driver management**

- A. WebDriverManager.
- B. Selenium Manager.
- C. Manually managed browser drivers.
- D. Remote WebDriver/Grid only.
- E. Combination depending on environment.

#### **18. Parallel execution**

- A. No parallel execution initially.
- B. Parallel methods.
- C. Parallel classes.
- D. Parallel tests/suites with isolated WebDriver instances.
- E. High-scale parallel execution through Selenium Grid/cloud.

#### **19. Expected parallel scale**

- A. 2–5 concurrent browsers.
- B. 6–20 concurrent browsers.
- C. 21–50 concurrent browsers.
- D. 50+ concurrent browsers.

E. Unknown; framework should be designed to scale dynamically.

## **20. Thread safety**

- A. ThreadLocal WebDriver.
- B. Dependency-injected driver/session management.
- C. Framework-managed driver factory with thread isolation.
- D. Other.

## **21. Test data source**

- A. Excel.
- B. CSV.
- C. JSON.
- D. Database.
- E. Combination of multiple sources.
- F. Generated/dynamic test data.

## **22. Data-driven execution model**

- A. TestNG DataProvider.
- B. External data mapped to TestNG DataProvider.
- C. Data-driven tests selected by environment/configuration.
- D. Combination of A+B+C.

## **23. Test data volume**

- A. Small — Less than 100 records/test run.
- B. Medium — 100–10,000 records.
- C. Large — 10,000+ records.
- D. Very large/dynamic — Millions or generated datasets.

## **24. Test data isolation**

- A. Shared static test data.
- B. Unique data per test.
- C. Unique data per thread/browser.
- D. Dynamic data creation and cleanup.
- E. Combination.

## **25. Environment management**

- A. Dev + QA.
- B. QA + UAT.
- C. Dev + QA + UAT + Stage/Pre-prod.
- D. Multiple environments controlled through configuration.
- E. Other.

## **26. Configuration management**

- A. properties files.

- B. YAML/JSON configuration.
- C. Environment variables + configuration files.
- D. CI/CD variables/secrets + configuration files.
- E. Combination.

## **27. Secrets management**

- A. Local properties for non-production credentials only.
- B. CI/CD secret variables.
- C. Enterprise secret manager/vault.
- D. Cloud secret-management service.
- E. Combination with no secrets committed to Git.

## **28. Authentication**

- A. Username/password UI login.
- B. SSO.
- C. OAuth/token-based authentication.
- D. Multiple authentication mechanisms.
- E. Other.

## **29. Login strategy**

- A. Perform UI login before each test.
- B. Login once per test class.
- C. Reuse authenticated session where safe.
- D. Environment-specific authentication strategy.
- E. Combination.

## **30. Test setup and teardown**

- A. TestNG @BeforeMethod/@AfterMethod.
- B. @BeforeClass/@AfterClass where appropriate.
- C. Suite-level setup/cleanup.
- D. Combination based on resource lifecycle.

## **31. Failure handling**

- A. Fail immediately and continue remaining tests.
- B. Retry failed tests automatically.
- C. Retry only known transient failures.
- D. No automatic retry; investigate every failure.
- E. Configurable retry policy by environment/test type.

## **32. Screenshot strategy**

- A. Screenshot only on failure.
- B. Screenshot on failure + selected checkpoints.
- C. Screenshot for every major test step.

D. Configurable screenshot policy.

### **33. Browser logs / artifacts**

A. Screenshots only.

B. Screenshots + browser console logs.

C. Screenshots + console logs + HTML/page source.

D. All relevant artifacts including video where infrastructure supports it.

### **34. Reporting**

A. TestNG default reports.

B. ExtentReports.

C. Allure.

D. Enterprise/custom reporting.

E. Multiple reporting outputs.

### **35. Logging framework**

A. SLF4J + Logback.

B. SLF4J + Log4j2.

C. Other logging framework.

D. No specific preference; recommend an industry-standard option.

### **36. CI/CD platform**

A. Jenkins.

B. GitHub Actions.

C. Azure DevOps Pipelines.

D. GitLab CI/CD.

E. Other.

F. Multiple CI/CD platforms.

### **37. Source control**

A. GitHub.

B. GitLab.

C. Azure Repos.

D. Bitbucket.

E. Other.

### **38. Containerization**

A. No Docker requirement.

B. Docker for test execution.

C. Docker + Selenium Grid.

D. Docker/Kubernetes-based scalable execution.

### **39. Selenium Grid / cloud execution**

A. Local execution only.

- B. Self-hosted Selenium Grid.
- C. Selenium Grid in Docker.
- D. Cloud provider such as BrowserStack/Sauce Labs/LambdaTest.
- E. Support both local and remote execution.

#### **40. API/database support**

- A. UI automation only.
- B. UI + API calls for setup/cleanup.
- C. UI + database validation.
- D. UI + API + database integration.
- E. Other.

#### **41. Database technology**

- A. PostgreSQL.
- B. MySQL.
- C. Oracle.
- D. SQL Server.
- E. Multiple databases.
- F. No database validation.

#### **42. API automation library**

- A. Rest Assured.
- B. Apache HttpClient/Java HTTP client.
- C. Other API library.
- D. No API automation required.

#### **43. Performance/non-functional scope**

- A. Functional UI only.
- B. Basic UI performance measurements.
- C. Separate performance framework/tool; Selenium remains functional-focused.
- D. Framework should integrate functional and non-functional test execution.

#### **44. Test execution selection**

- A. Run all tests.
- B. Run by TestNG group/tag.
- C. Run by feature/module.
- D. Run by environment and test type.
- E. Dynamic selection through CI/CD parameters.

#### **45. Configuration-driven execution**

- A. Command-line parameters.
- B. Maven profiles.
- C. TestNG XML suites.

- D. Environment variables + Maven profiles + TestNG XML.
- E. Other.

#### **46. Test naming and traceability**

- A. TestNG method names only.
- B. Test IDs mapped to requirements/defects.
- C. Test IDs + requirement IDs + CI/report integration.
- D. Enterprise test-management integration.

#### **47. Requirement/test management integration**

- A. No integration.
- B. Jira.
- C. Azure DevOps Test Plans/Boards.
- D. Other test-management tool.
- E. Multiple tools.

#### **48. Code quality**

- A. Basic code review.
- B. Checkstyle/PMD/SpotBugs.
- C. SonarQube/SonarCloud.
- D. Static analysis + quality gates in CI/CD.

#### **49. Dependency/version governance**

- A. Manually maintain dependencies.
- B. Pinned versions with scheduled upgrades.
- C. Dependency scanning and automated update process.
- D. Enterprise dependency governance.

#### **50. Framework extensibility**

- A. Optimize for current application only.
- B. Reusable within the current project.
- C. Reusable across multiple applications.
- D. Build as a long-term enterprise automation platform.

#### **51. Custom utilities required**

- A. Basic browser/wait utilities.
- B. Browser + waits + screenshots + file + data utilities.
- C. Comprehensive reusable utility layer.
- D. Utilities should be plugin/extensible.

#### **52. Design pattern expectations**

- A. POM only.
- B. POM + Factory.
- C. POM + Factory + Strategy/Builder where useful.



- D. SOLID + appropriate GoF patterns + clean architecture.
- E. No preference; recommend based on requirements.

### **53. Dependency injection**

- A. No DI; simple framework.
- B. Use TestNG features and framework factories.
- C. Use a DI framework such as Guice/Spring where justified.
- D. No preference; choose the simplest scalable approach.

### **54. Cross-browser data-driven matrix**

- A. Same tests on one browser.
- B. Same tests across selected browsers.
- C. Different data sets × different browsers × environments.
- D. Fully configurable execution matrix in CI/CD.

### **55. Execution time target**

- A. No specific target.
- B. Smoke suite under 10 minutes.
- C. Regression suite under 30 minutes.
- D. Regression suite under 1 hour.
- E. Maximum parallelization; optimize for the shortest practical time.

### **56. Flaky test management**

- A. Fix flakes manually when discovered.
- B. Track and retry transient failures.
- C. Quarantine flaky tests with reporting.
- D. Automated flaky-test detection, quarantine, metrics, and remediation workflow.

### **57. Maintainability expectations**

- A. Simple and easy for a small team.
- B. Maintainable for a medium QA team.
- C. Highly maintainable for multiple automation engineers.
- D. Enterprise-grade governance, standards, extensibility, and documentation.

### **58. Team skill level**

- A. Beginner Java/Selenium team.
- B. Intermediate Selenium team.
- C. Senior automation/SDET team.
- D. Mixed-skill team; framework must be easy to onboard.

### **59. Documentation requirements**

- A. README only.
- B. README + framework usage guide.
- C. Architecture + coding standards + onboarding + troubleshooting.

**D.** Complete technical documentation and operational runbook.

## **60. Final architecture priority**

**A.** Simplicity — Avoid unnecessary abstraction.

**B.** Maintainability — Make changes easy and safe.

**C.** Scalability — Support parallel, distributed, data-driven execution.

**D.** Reusability — Build for multiple applications/projects.

**E.** Enterprise readiness — Balance all of the above with governance and CI/CD.